

KERN EVENT STORE

Specifica tecnica per la prima versione production-ready single-node

Java · RocksDB · Spring Boot · Armeria · gRPC/HTTP

Versione 0.1 | Draft di implementazione | Luglio 2026

Documento autonomo: il prodotto non dipende né conosce il framework delle Decision.

Controllo del documento

Campo	Valore
Prodotto	Kern Event Store
Versione specifica	0.1
Stato	Draft implementabile
Obiettivo	Prima versione single-node utilizzabile in produzione
Tecnologie	Java, RocksDB/RocksJava, Spring Boot, Armeria, gRPC, HTTP/JSON, Micrometer/OpenTelemetry
Fuori perimetro	Framework delle Decision, clustering, sharding, multi-leader, query language arbitrario

Decisioni architetturali principali

ID	Decisione	Motivazione pragmatica
ADR-001	Server autonomo single-node	Riduce il rischio e consente di rendere solide durability, recovery e semantica prima della distribuzione.
ADR-002	RocksDB come storage engine	Fornisce WAL, WriteBatch atomici, ordinamento delle chiavi, snapshot, backup/checkpoint e buone prestazioni su SSD.
ADR-003	Single append coordinator	Semplifica posizione globale, DCB multi-key, idempotenza e group commit senza sacrificare il throughput iniziale.
ADR-004	Spring Boot come control plane	Configura lifecycle, dependency wiring, Actuator, sicurezza e amministrazione senza contaminare il core.
ADR-005	Armeria come data plane	Supporta HTTP/2, gRPC e streaming con controllo esplicito degli executor e backpressure.
ADR-006	Protobuf per API primaria	Contratto binario stabile, client generation e overhead ridotto; HTTP/JSON resta disponibile per operazioni e debug.
ADR-007	Eventi opachi allo store	Il server non conosce classi o semantica di dominio; conserva payload e metadata con content type e schema reference.
ADR-008	Osservabilità come capability di prodotto	Append, conflitti DCB, subscription, recovery e storage devono essere

ID	Decisione	Motivazione pragmatica
		diagnosticabili senza leggere file interni.

Indice

1. 1. Scopo, principi e perimetro
2. 2. Requisiti funzionali e non funzionali
3. 3. Modello concettuale e semantica
4. 4. Architettura del sistema
5. 5. API pubblica e protocolli
6. 6. Layout RocksDB e formato delle chiavi
7. 7. Percorso di append e consistenza
8. 8. Lettura, query e observation
9. 9. Subscription e projection feed
10. 10. Idempotenza e gestione duplicati
11. 11. Durability, recovery, backup e restore
12. 12. Sicurezza e multi-tenancy
13. 13. Osservabilità, monitoring e diagnostica
14. 14. Configurazione e operatività
15. 15. Prototipi di implementazione Java
16. 16. Testing e quality gates
17. 17. Prestazioni e benchmark
18. 18. Packaging e deployment
19. 19. Roadmap evolutiva
20. 20. Rischi, limiti e criteri di accettazione
21. Appendici

1. Scopo, principi e perimetro

1.1 Scopo

Kern Event Store è un server autonomo per la persistenza append-only di eventi. La prima versione deve supportare sia l'Event Sourcing tradizionale basato su stream/subject e expected revision, sia Dynamic Consistency Boundaries (DCB) key-based, nelle quali un append è accettato soltanto se un insieme dinamico di consistency key conserva le revisioni osservate dal client.

Il prodotto non conosce Command, Decision, Aggregate, Invariant o Notification. Espone primitive generiche: eventi, stream/subject, tag, posizioni, condizioni di append, consistency key, idempotenza, lettura e subscription.

1.2 Principi

- Persist first: un evento è visibile soltanto dopo un commit durevole e atomico.
- Explicit guarantees: ogni API dichiara ordine, atomicità, delivery e durability; nessuna garanzia implicita.
- Events are immutable: gli eventi committati non vengono aggiornati o cancellati dalle API ordinarie.
- Derived indexes are rebuildable: il log degli eventi è autorevole; gli indici secondari devono poter essere verificati o ricostruiti.
- At-least-once consumption: le subscription possono rideliverare; il consumer gestisce checkpoint/idempotenza.
- Bounded resources: code, buffer, batch e connessioni hanno limiti configurabili e producono backpressure.
- Operational transparency: ogni append e conflitto deve essere correlabile, misurabile e ispezionabile.
- Protocol independence: il core non dipende da Spring, Armeria, gRPC o Protobuf.

1.3 Obiettivi della v1

- Append atomico di uno o più eventi con posizione globale monotona.
- Expected revision per stream/subject.
- DCB key-based multi-key atomica nel singolo nodo.
- Idempotency key con restituzione del risultato originario.
- Lettura globale, per subject, per tipo e per tag esatti.
- Subscription streaming da posizione con reconnect e checkpoint esterno.
- Backup, restore, verify e rebuild degli indici.
- gRPC come protocollo primario e HTTP/JSON amministrativo/diagnostico.

- Metriche, tracing, structured logging, audit append e strumenti di diagnosi.

1.4 Non-obiettivi della v1

- Clustering o failover automatico.
- Replica sincrona o asincrona nativa.
- Sharding e partizionamento distribuito.
- DCB query-based con predicate arbitrari.
- Projection server-side o funzioni utente.
- SQL/EventQL.
- Retention automatica degli eventi di dominio.
- Transazioni tra più istanze dello store.
- Hard real-time.

2. Requisiti funzionali e non funzionali

2.1 Requisiti funzionali

ID	Requisito	Priorità
F-001	Append batch atomico con ordering interno stabile	Must
F-002	Posizione globale monotona assegnata dal server	Must
F-003	Expected revision per subject, inclusi NO_STREAM e ANY	Must
F-004	DCB key-based su insieme dinamico di chiavi/revisioni	Must
F-005	Idempotency key e replay del risultato	Must
F-006	Lettura globale paginata da posizione	Must
F-007	Lettura ordinata per subject e revisione	Must
F-008	Filtri per tipo e tag esatti	Must
F-009	Streaming subscription da posizione	Must
F-010	Checkpoint consumer esterno e cursor opachi	Must
F-011	Health/readiness/liveness e stato storage	Must
F-012	Backup/restore/verify	Must
F-013	Audit diagnostico degli append e conflitti	Must
F-014	TLS e autenticazione	Must
F-015	Multi-tenancy logica mediante	Should

ID	Requisito	Priorità
	namespace	
F-016	Persistent subscription server-side	Future

2.2 Requisiti non funzionali

Area	Requisito iniziale
Durability	ACK durable solo dopo WAL sync, salvo profilo esplicitamente relaxed.
Atomicità	Evento, indici, revisioni DCB e record di idempotenza nello stesso WriteBatch.
Disponibilità	Single-node; restart automatico e recovery deterministico.
Prestazioni	Target da validare con benchmark: almeno migliaia di append/s su SSD consumer/server, con p99 documentato.
Scalabilità	Verticale nella v1; letture/subscription concorrenti, scrittura coordinata.
Compatibilità	Formato on-disk e protocollo versionati; upgrade con migration plan.
Osservabilità	Metriche RED/USE, tracing OpenTelemetry, log JSON, endpoint diagnostici.
Sicurezza	TLS, authn, authorization per namespace e admin, audit accessi.
Operabilità	Backup testabile, disk guard, graceful shutdown, configuration validation.

3. Modello concettuale e semantica

3.1 Event

```
public record EventData(
    UUID id,
    String type,
    String subject,
    String contentType,
    String schema,
    byte[] payload,
    byte[] metadata,
    Map<String, String> tags
) {
    public EventData {
        Objects.requireNonNull(id);
        requireText(type, "type");
        requireText(subject, "subject");
        requireText(contentType, "contentType");
        payload = payload.clone();
        metadata = metadata == null ? new byte[0] : metadata.clone();
        tags = tags == null ? Map.of() : Map.copyOf(tags);
    }
}
```

```
}
}
```

Lo store tratta payload e metadata come byte opachi. `type`, `subject` e tag sono indicizzabili; `schema` è un riferimento applicativo e non viene interpretato dal server.

3.2 Stored event ed envelope

```
public record StoredEvent(
    long position,
    long subjectRevision,
    UUID id,
    String type,
    String subject,
    Instant occurredAt,
    String contentType,
    String schema,
    byte[] payload,
    byte[] metadata,
    Map<String, String> tags,
    String tenant
) {}
```

3.3 Posizione globale

La posizione globale è un `long` strettamente crescente assegnato dall'append coordinator. Identifica l'ordine di commit nel singolo nodo. Posizioni mancanti non sono ammesse nel percorso ordinario; in caso di recovery il contatore viene ricostruito dall'ultimo evento committato e dal metadata store.

3.4 Subject e stream

Nella v1 `subject` è anche l'identificatore dello stream. Ogni evento possiede una `subjectRevision` crescente e contigua a partire da 1. Il termine subject è preferito perché non impone un aggregate model, ma l'API espone expected revision equivalente a un event store tradizionale.

3.5 Consistency key e DCB

```
public record ConsistencyKey(String value) {
    public ConsistencyKey {
        requireText(value, "value");
    }
}

public record ExpectedConsistency(
    Map<ConsistencyKey, Long> revisions
) implements AppendCondition {
    public ExpectedConsistency {
        revisions = Map.copyOf(revisions);
    }
}
```

Una consistency key è una coordinata logica scelta dal client, ad esempio `student:S1`, `course:C1` o `enrollment:S1:C1`. Lo store mantiene una revisione per ciascuna chiave. Un append DCB viene accettato solo se tutte le revisioni attuali coincidono con quelle attese; le chiavi dichiarate come toccate dall'append vengono poi incrementate atomicamente.

Vincolo v1: Le DCB sono key-based. Lo store non valuta predicate arbitrari sul contenuto degli eventi. Questa scelta rende la semantica verificabile, indicizzabile e implementabile in modo atomico con un single writer.

3.6 Condizioni di append

```
public sealed interface AppendCondition
    permits AnyAppend, ExpectedSubjectRevision, ExpectedConsistency {
}

public enum SubjectExpectation {
    ANY,
    NO_STREAM
}

public record ExpectedSubjectRevision(
    String subject,
```

```
        Long revision,
        SubjectExpectation expectation
    ) implements AppendCondition {}
```

3.7 Esito dell'append

```
public record AppendResult(
    long fromPosition,
    long toPosition,
    Map<String, Long> subjectRevisions,
    Map<ConsistencyKey, Long> consistencyRevisions,
    boolean replayedFromIdempotency
) {}
```

4. Architettura del sistema

```
Clients
  | gRPC / HTTP2 / HTTP JSON
  v
Armeria Data Plane
  | validation + authentication + mapping
  v
Event Store Core
  |-- AppendCoordinator --> RocksEventStorage
  |-- EventReader -----> RocksEventStorage
  |-- SubscriptionEngine -> EventReader
  |-- DiagnosticService -> audit/index/storage views
  v
RocksDB
  |-- authoritative events
  |-- subject/type/tag indexes
  |-- consistency revisions
  |-- idempotency
  |-- metadata/audit

Spring Boot Control Plane
  |-- configuration and lifecycle
  |-- Actuator, Micrometer, OpenTelemetry
  |-- admin API, backup, verify, security
```

4.1 Componenti

Componente	Responsabilità
Armeria data plane	gRPC, HTTP/2, streaming, timeout, backpressure, request context.
Spring Boot shell	Bootstrap, configurazione, bean wiring, lifecycle, Actuator e sicurezza amministrativa.
AppendCoordinator	Serializza append, verifica condizioni, assegna posizioni, raggruppa WriteBatch, completa future.
ConsistencyEngine	Valida expected subject revision e DCB key-based.
RocksEventStorage	Codec chiavi/valori, column family, letture e scritture atomiche.
EventReader	Query strutturate e cursor.
SubscriptionEngine	Tail durable da posizione, buffering e wake-up.
BackupManager	Checkpoint, manifest backup, verifica e restore.
DiagnosticService	Explain append, conflict report, event lineage, index verify.
Observability	Metriche, trace, structured logs, JFR hooks e diagnostics

Componente	Responsabilità
	snapshots.

4.2 Dipendenze ammesse

`kern-core` non dipende da Spring, Armeria, Protobuf o RocksDB. `kern-rocks` implementa le SPI del core. `kern-server` assembla Spring Boot e Armeria. Questa separazione rende il core benchmarkabile e consente adapter futuri.

4.3 Threading model

```
Armeria event loops
  -> validate + enqueue only
Bounded MPSC append queue
  -> one append coordinator thread
Read executor
  -> concurrent RocksDB reads and scans
Subscription executor
  -> cursor advancement and network backpressure
Maintenance executor
  -> backup, verify, index rebuild, compaction administration
```

Nessun fsync, scan RocksDB o serializzazione pesante deve essere eseguito sull'event loop Armeria. La coda di append è limitata: quando piena il server risponde `RESOURCE\_EXHAUSTED`/HTTP 429 invece di accumulare memoria senza controllo.

5. API pubblica e protocolli

5.1 Servizio gRPC

```
service EventStoreService {
  rpc Append(AppendRequest) returns (AppendResponse);
  rpc Read(ReadRequest) returns (ReadResponse);
  rpc Observe(ObserveRequest) returns (stream EventRecord);
  rpc GetEvent(GetEventRequest) returns (EventRecord);
  rpc GetRevisions(GetRevisionsRequest) returns (RevisionsResponse);
}

service AdministrationService {
  rpc CreateBackup(CreateBackupRequest) returns (OperationResponse);
  rpc Verify(VerifyRequest) returns (stream VerificationProgress);
  rpc GetDiagnostics(DiagnosticsRequest) returns (DiagnosticsResponse);
}
```

5.2 Prototipo Protobuf append

```
message AppendRequest {
  string namespace = 1;
  string idempotency_key = 2;
  repeated ProposedEvent events = 3;
  oneof condition {
    AnyAppend any = 4;
    ExpectedSubjectRevision expected_subject = 5;
    ExpectedConsistency expected_consistency = 6;
  }
  repeated string touched_consistency_keys = 7;
  Durability durability = 8;
}

message ProposedEvent {
  string id = 1;
  string type = 2;
  string subject = 3;
  string content_type = 4;
  string schema = 5;
  bytes payload = 6;
  bytes metadata = 7;
  map<string,string> tags = 8;
}

enum Durability {
  DURABILITY_UNSPECIFIED = 0;
```

```

DURABLE = 1;
RELAXED = 2;
}

```

5.3 Error model

Errore dominio protocollo	gRPC	HTTP	Retry
InvalidRequest	INVALID_ARGUMENT	400	No
AuthenticationFailed	UNAUTHENTICATED	401	No
Forbidden	PERMISSION_DENIED	403	No
ConsistencyConflict	ABORTED	409	Sì, dopo nuova observation
IdempotencyConflict	ALREADY_EXISTS	409	No; stessa chiave con payload diverso
Overloaded	RESOURCE_EXHAUSTED	429	Sì, con backoff
StorageUnavailable	UNAVAILABLE	503	Sì
DiskSpaceGuard	FAILED_PRECONDITION	507	Dopo intervento operativo
InternalCorruption	DATA_LOSS	500	No, escalation

5.4 HTTP amministrativo e diagnostico

```

GET /actuator/health
GET /actuator/prometheus
GET /v1/diagnostics/storage
GET /v1/diagnostics/append/{idempotencyKey}
GET /v1/diagnostics/conflicts/{conflictId}
GET /v1/events/{eventId}
POST /v1/admin/backups
POST /v1/admin/verify
POST /v1/admin/rebuild-indexes
POST /v1/admin/compact

```

5.5 Limiti di request

Parametro	Default iniziale	Comportamento
maxEventsPerAppend	256	Rifiuto validazione
maxAppendBytes	4 MiB	Rifiuto validazione
maxPayloadBytes	1 MiB	Rifiuto per evento
maxTagsPerEvent	32	Rifiuto validazione
maxTagLength	256 byte UTF-8	Rifiuto validazione
maxSubjectLength	512 byte UTF-8	Rifiuto validazione
appendTimeout	10 s	Deadline gRPC/HTTP

Parametro	Default iniziale	Comportamento
observeBuffer	1024 eventi	Backpressure/disconnessione consumer lento

6. Layout RocksDB e formato delle chiavi

6.1 Column family

Column family	Autorevole	Chiave	Valore
events	Sì	namespace + globalPosition	StoredEvent binary record
subjects	No	namespace + subject + revision	globalPosition
event_ids	No	namespace + event UUID	globalPosition
types	No	namespace + type + globalPosition	empty/reference
tags	No	namespace + tagName + tagValue + position	empty/reference
subject_heads	Sì	namespace + subject	latestRevision + latestPosition
consistency	Sì	namespace + consistencyKey	revision + latestPosition
idempotency	Sì	namespace + idempotencyKey	requestDigest + AppendResult
system	Sì	well-known keys	formatVersion, nextPosition, cleanShutdown
audit	Sì diagnostica	timestamp/sequence or request id	append/conflict diagnostic record
subscriptions	Future	subscription + partition	ack state

6.2 Ordinamento binario delle chiavi

Le chiavi numeriche usano big-endian unsigned ordering per preservare l'ordine lessicografico. Le stringhe sono codificate UTF-8 con length prefix, non con separatori ambigui. Il namespace è sempre il primo componente per isolare scan e autorizzazione.

```
final class KeyCodec {
    byte[] eventKey(String namespace, long position) {
        return KeyBuilder.create()
            .utf8(namespace)
            .byteValue((byte) 0x01)
            .longBigEndian(position)
            .build();
    }

    byte[] subjectKey(String namespace, String subject, long revision) {
        return KeyBuilder.create()
            .utf8(namespace)
            .utf8(subject)
            .byteValue((byte) 0x02)
            .longBigEndian(revision)
            .build();
    }
}
```

```

        .utf8(namespace)
        .byteValue((byte) 0x02)
        .utf8(subject)
        .longBigEndian(revision)
        .build();
    }
}

```

6.3 Formato del record

Il formato interno deve essere versionato e indipendente dalla Java serialization. Si raccomanda Protobuf interno o codec binario esplicito con: magic, version, lengths, payload, metadata e checksum CRC32C. Il checksum applicativo facilita diagnostica e verify oltre alle protezioni del motore.

```

record EventRecordHeader(
    int formatVersion,
    long position,
    long subjectRevision,
    long timestampEpochMicros,
    int payloadLength,
    int metadataLength,
    int checksum
) {}

```

6.4 WriteBatch atomico

```

try (WriteBatch batch = new WriteBatch()) {
    batch.put(eventsCf, keyCodec.eventKey(ns, position), encodedEvent);
    batch.put(subjectsCf, keyCodec.subjectKey(ns, subject, revision), longBytes(position));
    batch.put(eventIdsCf, keyCodec.eventIdKey(ns, id), longBytes(position));
    batch.put(subjectHeadsCf, keyCodec.subjectHeadKey(ns, subject), encodeHead(revision, position));
    batch.put(consistencyCf, keyCodec.consistencyKey(ns, key), encodeRevision(nextRevision, position));
    batch.put(idempotencyCf, keyCodec.idempotencyKey(ns, token), encodedResult);
    batch.put(systemCf, NEXT_POSITION, longBytes(nextPosition));
    rocks.write(writeOptions, batch);
}

```

7. Percorso di append e consistenza

7.1 Sequenza

22. Validare dimensioni, identificatori, duplicati nel batch e autorizzazione sul namespace.
23. Calcolare un digest canonico della richiesta per l'idempotenza.
24. Accodare la richiesta al bounded append coordinator.
25. Nel writer, verificare record idempotency già esistente.
26. Leggere e validare expected subject revision o expected consistency revisions.
27. Assegnare posizioni globali e revisioni subject.
28. Costruire eventi e indici nel medesimo WriteBatch.
29. Incrementare tutte le touched consistency key in ordine canonico.
30. Scrivere audit record e risultato idempotente.
31. Eseguire RocksDB write con WAL sync secondo durability.
32. Pubblicare il nuovo high-watermark ai subscription waiter.
33. Completare la future e il trace.

7.2 API core

```

public interface EventStore {
    CompletionStage<AppendResult> append(AppendRequest request);
    EventPage read(EventQuery query);
    Optional<StoredEvent> get(UUID id);
    Revisions getRevisions(RevisionQuery query);
    EventSubscription observe(ObserveRequest request, EventConsumer consumer);
}

public record AppendRequest(
    String namespace,
    String idempotencyKey,
    List<EventData> events,
    AppendCondition condition,
    Set<ConsistencyKey> touchedConsistencyKeys,
    Durability durability
) {}

```

7.3 AppendCoordinator

```
public final class AppendCoordinator implements AutoCloseable {
    private final BlockingQueue<PendingAppend> queue;
    private final Thread writer;
    private final AtomicBoolean running = new AtomicBoolean(true);

    public CompletionStage<AppendResult> append(AppendRequest request) {
        var future = new CompletableFuture<AppendResult>();
        if (!queue.offer(new PendingAppend(request, future))) {
            future.completeExceptionally(new StoreOverloadedException());
        }
        return future;
    }

    private void runWriter() {
        while (running.get() || !queue.isEmpty()) {
            try {
                PendingAppend first = queue.poll(100, TimeUnit.MILLISECONDS);
                if (first == null) continue;
                List<PendingAppend> group = drainCompatible(first);
                storage.commit(group);
            } catch (Throwable failure) {
                failCurrentGroup(failure);
            }
        }
    }
}
```

7.4 Group commit

La prima implementazione può committare una richiesta per WriteBatch. Il group commit è una ottimizzazione prevista fin dall'API: richieste con la stessa durability possono essere consolidate in un singolo WAL sync, pur mantenendo esiti e condizioni indipendenti. Una condizione fallita non deve invalidare le richieste successive; il batch logico può essere prevalidato in memoria sotto il single writer.

7.5 Expected revision tradizionale

```
long current = heads.readRevision(namespace, expected.subject());
boolean matches = switch (expected.expectation()) {
    case ANY -> true;
    case NO_STREAM -> current == 0L;
};
if (expected.revision() != null) {
    matches = current == expected.revision();
}
if (!matches) {
    throw SubjectRevisionConflict.of(expected, current);
}
```

7.6 DCB key-based

```
void verifyConsistency(ExpectedConsistency expected, String namespace) {
    for (var entry : sorted(expected.revisions())) {
        long actual = consistency.readRevision(namespace, entry.getKey());
        if (actual != entry.getValue()) {
            throw new ConsistencyConflict(
                entry.getKey(), entry.getValue(), actual);
        }
    }
}

void advanceTouchedKeys(Set<ConsistencyKey> touched, long position, WriteBatch batch) {
    for (ConsistencyKey key : sorted(touched)) {
        long current = consistency.readRevision(namespace, key);
        consistency.put(batch, namespace, key, current + 1, position);
    }
}
```

7.7 Idempotenza

La stessa `idempotencyKey` con digest identico restituisce l'AppendResult originario. La stessa chiave con richiesta differente produce `IdempotencyConflict`. Il digest include namespace, eventi nell'ordine, condizione, touched key e durability semantica; esclude metadata di trasporto come trace ID.

7.8 Semantica di ACK

Durability	WriteOptions	Garanzia
DURABLE	WAL enabled, sync=true	Risposta dopo richiesta di sincronizzazione del WAL. È il default production.
RELAXED	WAL enabled, sync=false	Il commit è nel WAL/page cache ma può essere perso in un guasto di sistema; solo opt-in.
MEMORY_ONLY	Non supportato in production	Eventuale profilo test futuro, non esposto dal server standard.

8. Lettura, query e observation

8.1 EventQuery

```
public record EventQuery(
    String namespace,
    Optional<String> subject,
    Set<String> types,
    Map<String, String> exactTags,
    long afterPosition,
    int limit,
    Direction direction
) {}
```

La v1 non supporta AND/OR arbitrari. `subject`, tipi e tag esatti vengono combinati con AND; più tipi sono OR tra loro. Il planner sceglie l'indice più selettivo disponibile e verifica i restanti predicate sul record evento.

8.2 Paginazione

```
public record EventPage(
    List<StoredEvent> events,
    String continuationToken,
    long highWatermark
) {}
```

Il continuation token è firmato o codificato come struttura opaca contenente query hash, indice scelto e ultima posizione. Il client non costruisce direttamente chiavi RocksDB.

8.3 Lettura coerente multi-column-family

Per query che attraversano più column family si usa un RocksDB Snapshot per ottenere una vista point-in-time coerente. Lo snapshot è limitato alla durata della richiesta e viene sempre chiuso. Non viene restituito al client e non sopravvive a un restart.

```
try (Snapshot snapshot = rocks.getSnapshot();
    ReadOptions options = new ReadOptions().setSnapshot(snapshot);
    RocksIterator iterator = rocks.newIterator(eventsCf, options)) {
    iterator.seek(keyCodec.eventKey(namespace, after + 1));
    // collect up to limit and close resources deterministically
}
```

8.4 Revision observation

```
public record Revisions(
    Map<String, Long> subjects,
    Map<ConsistencyKey, Long> consistencyKeys,
    long observedAtPosition
) {}

public record RevisionQuery(
    String namespace,
    Set<String> subjects,
```

```
    Set<ConsistencyKey> consistencyKeys
  ) {}
```

Questa API permette al client di ottenere token di consistenza senza leggere payload. La lettura avviene sotto snapshot e include l'high-watermark osservato per diagnostica e tracing.

9. Subscription e projection feed

9.1 Semantica

`Observe` è una subscription stateless da posizione. Il server invia eventi committati con posizione maggiore di `afterPosition`, applicando gli stessi filtri della query. Il client salva il checkpoint nel proprio storage. Dopo reconnect riparte dall'ultima posizione committata localmente.

9.2 SubscriptionEngine

```
public interface EventSubscription extends AutoCloseable {
    SubscriptionId id();
    CompletionStage<Void> completion();
    void cancel();
}

public final class DefaultSubscriptionEngine {
    private final HighWatermarkSignal signal;
    private final EventReader reader;

    EventSubscription observe(ObserveRequest request, EventConsumer consumer) {
        return executor.start(() -> tail(request, consumer));
    }

    private void tail(ObserveRequest request, EventConsumer consumer) {
        long cursor = request.afterPosition();
        while (!Thread.currentThread().isInterrupted()) {
            EventPage page = reader.read(request.query().after(cursor));
            for (StoredEvent event : page.events()) {
                consumer.accept(event); // honors bounded downstream demand
                cursor = event.position();
            }
            signal.awaitAbove(cursor, request.heartbeatInterval());
        }
    }
}
```

9.3 Backpressure

Armeria/gRPC deve propagare demand e cancellation. Ogni subscription ha un buffer limitato; se il consumer non legge entro `slowConsumerTimeout`, il server termina la stream con errore esplicito e registra la posizione raggiunta nel log diagnostico. Non vengono mai accumulati eventi senza limite.

9.4 Ordering

- La subscription globale conserva l'ordine di posizione globale.
- Una query filtrata può saltare posizioni ma mantiene l'ordine relativo degli eventi restituiti.
- Non esiste parallelizzazione server-side della medesima subscription nella v1.
- La delivery è at-least-once rispetto al checkpoint del consumer; il server non promette exactly-once.

9.5 Wake-up

Il commit aggiorna un high-watermark atomico e segnala i waiter. Il segnale in memoria è solo un'ottimizzazione: dopo timeout o perdita del segnale, il reader interroga comunque il log dalla posizione corrente.

10. Idempotenza e gestione duplicati

10.1 Duplicati evento

`event.id` è unico nel namespace. Un batch contenente ID già esistente viene rifiutato, salvo che l'intera richiesta sia un replay idempotente riconosciuto dalla idempotency key. Non si tenta il merge di batch parzialmente duplicati.

10.2 Record idempotency

```
record IdempotencyRecord(
    byte[] requestDigest,
    AppendResult result,
    Instant committedAt,
    String principal,
    String traceId
) {}
```

I record di idempotenza sono parte della storia operativa. La v1 li conserva senza scadenza automatica; una futura policy di retention potrà essere introdotta soltanto con finestre documentate e senza compromettere retry tardivi.

11. Durability, recovery, backup e restore

11.1 Startup sequence

34. Acquisire un file lock esclusivo sulla data directory.
35. Aprire RocksDB con tutte le column family attese.
36. Validare `formatVersion` e configuration fingerprint compatibile.
37. Leggere il clean shutdown marker.
38. Eseguire quick verification di metadata, next position e ultimo evento.
39. Se shutdown non pulito, emettere recovery trace e verificare code/indici critici.
40. Impostare clean shutdown=false prima di accettare traffico.
41. Avviare Armeria e passare readiness a UP soltanto a recovery conclusa.

11.2 Graceful shutdown

```
public void stop() {
    readiness.refuseTraffic();
    armeria.stopAcceptingNewRequests();
    appendCoordinator.drain(shutdownTimeout);
    subscriptionEngine.stop();
    storage.flushWal(true);
    systemMetadata.markCleanShutdown();
    storage.close();
}
```

11.3 Backup

Il backup usa RocksDB Checkpoint/BackupEngine e produce un manifest esterno contenente store ID, format version, createdAt, high-watermark, checksum dei file e configurazione essenziale. Il backup è creato senza arrestare il server, ma la sua completion e verifica sono tracciate.

```
public record BackupManifest(
    UUID storeId,
    int formatVersion,
    long highWatermark,
    Instant createdAt,
    Map<String, String> fileChecksums
) {}

public BackupResult createBackup(Path target) {
    maintenanceLock.readLock().lock();
    try {
        storage.createCheckpoint(target.resolve("db"));
        BackupManifest manifest = verifier.buildManifest(target);
        manifestCodec.write(target.resolve("manifest.json"), manifest);
        verifier.verifyBackup(target);
        return new BackupResult(target, manifest);
    } finally {

```



```

    maintenanceLock.readLock().unlock();
}

```

11.4 Restore

Il restore avviene sempre su directory nuova. Il server verifica manifest e checksum, apre il database in modalità offline, esegue verify, quindi effettua uno switch atomico della data directory o richiede un riavvio controllato. Non si sovrascrive un database aperto.

11.5 Verify e rebuild

Modalità	Controlli
Quick	format, metadata, last position, column family presence, sample checksum
Full	scan eventi, checksum, monotonicità, subject revision, event ID uniqueness, consistency records
Indexes	confronto eventi vs subject/type/tag/event-id indexes
Rebuild	creazione di nuove CF/DB da log autorevole, validazione e switch offline

11.6 Disk guard

Il server monitora spazio libero e write stall. Sotto la soglia warning emette alert; sotto la soglia critical rifiuta nuovi append ma mantiene letture e diagnostica. L'obiettivo è evitare corruzioni o failure incontrollate per disco pieno.

12. Sicurezza e multi-tenancy

12.1 Autenticazione

La v1 supporta mTLS e/o bearer token JWT validato da Spring Security/Armeria decorator. Il principal viene inserito nel request context e nell'audit diagnostico, non nel payload evento salvo scelta del client.

12.2 Autorizzazione

Ruolo	Permessi
writer:<namespace>	Append e lettura revisioni nel namespace
reader:<namespace>	Read, get e observe
operator	Health dettagliato, diagnostics non sensibili, backup
admin	Restore, verify, rebuild, compact, configuration inspection

12.3 Namespace

Il namespace isola chiavi, indici, quota e autorizzazioni. Non fornisce isolamento fisico: tutte le namespace condividono lo stesso RocksDB e le stesse risorse. Per tenant ad alta criticità si raccomanda un'istanza dedicata.

12.4 Cifratura

TLS protegge il transito. La cifratura at-rest è demandata inizialmente al filesystem/volume. Una encryption envelope del payload è possibile lato client; una future versione potrà introdurre key management server-side, ma non deve bloccare la v1.

13. Osservabilità, monitoring e diagnostica

13.1 Obiettivo distintivo

L'Event Sourcing diventa difficile da debuggare quando eventi, causalità, revisioni e projection lag sono invisibili. Kern Event Store deve rendere osservabili non soltanto CPU e throughput, ma anche la semantica dell'append: cosa è stato osservato, quale condizione è fallita, quali revisioni erano attese e quali erano attuali, dove si trova un evento e quale subscriber è in ritardo.

13.2 Metriche

Metrica	Tipo	Tag controllati
kern.append.requests	counter	namespace, outcome, durability
kern.append.latency	timer	outcome, durability
kern.append.queue.depth	gauge	none
kern.append.batch.events	distribution	none
kern.append.conflicts	counter	conditionType, keyHashPrefix
kern.rocks.write.stall	gauge/counter	reason
kern.rocks.compaction.pending_bytes	gauge	columnFamily
kern.read.latency	timer	queryType, outcome
kern.read.events	distribution	queryType
kern.subscription.active	gauge	namespace
kern.subscription.lag	gauge	subscriptionIdHash
kern.subscription.slow_consumer	counter	namespace
kern.backup.duration	timer	outcome
kern.verify.errors	counter	category
kern.disk.free.bytes	gauge	volume
kern.store.high_watermark	gauge	namespace

Cardinalità: Non usare subject, eventId, idempotencyKey o consistencyKey complete come tag metrici. Questi valori vanno in log/trace/audit interrogabili; le metriche usano categorie o hash prefix limitati.

13.3 Tracing

Ogni append produce uno span `eventstore.append` con child span per queue wait, condition check, encode, RocksDB write/WAL sync e response. Gli attributi includono namespace, numero eventi, byte, durability, condition type, from/to position e outcome. Dati sensibili e payload non vengono registrati.

```
try (Observation.Scope scope = observationRegistry
    .observation("eventstore.append")
    .lowCardinalityKeyValue("condition", request.condition().type())
    .lowCardinalityKeyValue("durability", request.durability().name())
    .highCardinalityKeyValue("request.id", diagnosticRequestId)
    .start().openScope()) {
    return appendCoordinator.append(request);
}
```

13.4 Structured logging

```
{
  "event": "append_committed",
  "requestId": "req-...",
  "traceId": "...",
  "namespace": "education",
  "eventCount": 2,
  "fromPosition": 10421,
  "toPosition": 10422,
  "condition": "expected-consistency",
  "queueWaitMicros": 84,
  "walSyncMicros": 713,
  "totalMicros": 1120
}
```

Il livello INFO registra commit e failure aggregate; DEBUG può includere revisioni e indici; TRACE non deve includere payload per default. I conflitti DCB hanno un `conflictId` interrogabile.

13.5 Append diagnostic record

```
record AppendDiagnostic(
    String requestId,
    String idempotencyKeyHash,
    Instant receivedAt,
    Instant completedAt,
    String namespace,
    String conditionType,
    Map<String, Long> expectedRevisions,
    Map<String, Long> actualRevisions,
    Set<String> touchedKeys,
    AppendResult result,
    String outcome,
    String principal,
    String traceId,
    TimingBreakdown timing
) {}
```

Per privacy e dimensione, chiavi complete e subject possono essere conservati nel diagnostic store secondo policy configurabile; in ambienti sensibili si usa hashing con possibilità di correlazione. I record diagnostici non fanno parte del log di dominio e possono avere retention diversa.

13.6 Explain conflict

```
GET /v1/diagnostics/conflicts/cf-8127

{
  "condition": "expected-consistency",
  "failedKeys": [
    { "key": "course:C1", "expected": 42, "actual": 43, "latestPosition": 10420 }
  ],
  "observationHint": {
    "readAfter": 10420,
    "subjects": ["course:C1"]
  }
}
```

Questa capability rende il conflitto azionabile: il client può sapere quali key hanno invalidato l'append e da quale posizione rileggere, senza dover interpretare un generico 409.

13.7 Event inspector

- Ricerca evento per UUID o posizione.
- Visualizzazione header, subject revision, tag, schema, checksum e dimensioni.
- Navigazione precedente/successivo nello stesso subject.
- Visualizzazione degli append diagnostic correlati tramite request/trace metadata.
- Payload nascosto per default e mostrato solo a ruoli autorizzati.
- Export NDJSON di un intervallo per analisi offline.

13.8 Subscription lag

Sebbene il checkpoint sia esterno nella v1, il consumer può inviare periodicamente un heartbeat opzionale con `subscriptionName` e posizione applicata. Lo store conserva uno stato diagnostico volatile/persistibile e calcola lag rispetto all'high-watermark. Questo non modifica la semantica di delivery.

```
message ReportConsumerPositionRequest {
  string namespace = 1;
  string consumer_name = 2;
  int64 applied_position = 3;
  map<string,string> labels = 4;
}
```

13.9 Dashboard e alert minimi

Dashboard	Pannelli
Write path	RPS, p50/p95/p99, queue depth, conflicts, WAL sync, batch size, errors
Storage	disk free, SST size, memtable, block cache, compaction, stalls, WAL size
Read/subscription	read latency, active streams, lag, slow consumers, reconnect
Reliability	unclean restarts, verify failures, backup age, backup result, checksum errors
Capacity	events/day, bytes/day, growth projection, namespace usage

- Alert critical: disk free sotto soglia, DATA_LOSS, verify failure, backup troppo vecchio, write stall prolungato.
- Alert warning: p99 append sopra SLO, queue depth sostenuta, compaction backlog, consumer lag crescente.
- Alert info: unclean restart recuperato, configurazione cambiata, rebuild iniziato/completato.

13.10 Diagnostics snapshot

Un endpoint admin produce un pacchetto diagnostico senza payload: configurazione redatta, metric snapshot, RocksDB properties, thread dump, recent error/conflict summaries, backup state e versioni. Il pacchetto facilita supporto e incident response senza accesso diretto alla data directory.

14. Configurazione e operatività

14.1 Configurazione Spring

```
kern:
  event-store:
    id: primary
    data-directory: /var/lib/kern-event-store
    format-version: 1
    append:
      queue-capacity: 65536
```

```

max-events: 256
max-bytes: 4MB
group-commit-max-requests: 128
default-durability: durable
reads:
  threads: 8
  max-page-size: 1000
subscriptions:
  threads: 8
  max-buffered-events: 1024
  slow-consumer-timeout: 30s
storage:
  block-cache: 2GB
  write-buffer-size: 128MB
  max-background-jobs: 8
disk-guard:
  warning-free-bytes: 20GB
  critical-free-bytes: 5GB
diagnostics:
  retention: 30d
  store-full-keys: false
management:
  endpoints.web.exposure.include: health,info,prometheus,metrics,loggers

```

14.2 Validazione configurazione

Il server deve fallire all'avvio su directory non scrivibile, soglie incoerenti, memoria RocksDB superiore al budget, TLS obbligatorio senza certificati o format version incompatibile. Non deve avviarsi con fallback silenziosi.

14.3 Readiness e liveness

Probe	UP quando	DOWN quando
Liveness	processo ed append writer thread vivi	deadlock writer, fatal corruption, lifecycle bloccato
Readiness	storage aperto, recovery completata, disk guard non critical	startup/recovery, shutting down, disk critical, storage error
Health detail	include high-watermark, queue, disk, backup age, compaction	visibile solo a operator/admin

14.4 Manutenzione

Backup e verify possono girare online con limiti di I/O. Rebuild index e restore sono offline nella v1. La compaction manuale è un'operazione admin rate-limited e normalmente non necessaria; le policy automatiche RocksDB restano il default.

15. Prototipi di implementazione Java

15.1 Moduli Gradle/Maven

```

kern-event-store-parent
kern-event-store-api
kern-event-store-core
kern-event-store-rocks
kern-event-store-protocol
kern-event-store-armeria
kern-event-store-observability
kern-event-store-server
kern-event-store-client-java
kern-event-store-testkit
kern-event-store-benchmarks

```

15.2 SPI storage

```

public interface EventStorage extends AutoCloseable {
    CommitOutcome commit(List<PreparedAppend> appends, Durability durability);
}

```

```

EventPage read(EventQuery query, ReadSnapshot snapshot);
Revisions revisions(RevisionQuery query, ReadSnapshot snapshot);
Optional<StoredEvent> eventById(String namespace, UUID id, ReadSnapshot snapshot);
ReadSnapshot snapshot();
long highWatermark();
StorageDiagnostics diagnostics();
void flushWal(boolean sync);
}

```

15.3 Preparazione append

```

record PreparedAppend(
    AppendRequest request,
    byte[] requestDigest,
    CompletableFuture<AppendResult> future,
    String diagnosticRequestId,
    Instant receivedAt
) {}

final class AppendPreparer {
    PreparedAppend prepare(AppendRequest request) {
        validator.validate(request);
        byte[] digest = digestCodec.digestCanonical(request);
        return new PreparedAppend(
            request,
            digest,
            new CompletableFuture<>(),
            ids.nextRequestId(),
            clock.instant());
    }
}

```

15.4 RocksDB lifecycle

```

public final class RocksEventStorage implements EventStorage {
    static {
        RocksDB.loadLibrary();
    }

    private final RocksDB db;
    private final Map<ColumnFamily, ColumnFamilyHandle> handles;

    public static RocksEventStorage open(StoreConfiguration configuration) {
        List<ColumnFamilyDescriptor> descriptors = columnFamilies(configuration);
        List<ColumnFamilyHandle> handles = new ArrayList<>();
        DBOptions options = dbOptions(configuration);
        RocksDB db = RocksDB.open(options,
            configuration.dataDirectory().toString(), descriptors, handles);
        return new RocksEventStorage(db, mapHandles(descriptors, handles));
    }

    @Override
    public void close() {
        handles.values().forEach(ColumnFamilyHandle::close);
        db.close();
    }
}

```

15.5 Armeria server

```

@Configuration
public class ArmeriaConfiguration {
    @Bean
    ServerConfigurator eventStoreServices(EventStoreGrpcService grpc,
        AdministrationHttpService admin) {
        return builder -> builder
            .service(grpc)
            .serviceUnder("/v1/admin", admin)
            .decorator(TracingService.newDecorator())
            .decorator(MetricCollectingService.newDecorator())
            .requestTimeout(Duration.ofSeconds(10));
    }
}

```

15.6 gRPC append adapter

```

final class EventStoreGrpcService extends EventStoreServiceGrpc.EventStoreServiceImplBase {
    private final EventStore store;
    private final ProtocolMapper mapper;

    @Override

```

```

    public void append(ProtoAppendRequest input,
                      StreamObserver<ProtoAppendResponse> observer) {
        AppendRequest request;
        try {
            request = mapper.toDomain(input, ServiceRequestContext.current());
        } catch (RuntimeException invalid) {
            observer.onError(Status.INVALID_ARGUMENT
                .withDescription(invalid.getMessage()).asRuntimeException());
            return;
        }

        store.append(request).whenComplete((result, failure) -> {
            if (failure != null) {
                observer.onError(GrpcErrors.map(failure));
            } else {
                observer.onNext(mapper.toProto(result));
                observer.onCompleted();
            }
        });
    }
}

```

15.7 HTTP event inspector

```

@Get("/v1/events/:id")
public HttpResponse getEvent(ServiceRequestContext ctx, @Param String id) {
    Principal principal = authorizer.requireReader(ctx);
    return HttpResponse.from(
        CompletableFuture.supplyAsync(() -> inspector.find(UUID.fromString(id)), readExecutor)
            .thenApply(event -> HttpResponse.ofJson(event.redactedFor(principal))));
}

```

15.8 High-watermark signal

```

final class HighWatermarkSignal {
    private final AtomicLong highWatermark = new AtomicLong();
    private final ReentrantLock lock = new ReentrantLock();
    private final Condition advanced = lock.newCondition();

    void publish(long position) {
        highWatermark.accumulateAndGet(position, Math::max);
        lock.lock();
        try { advanced.signalAll(); }
        finally { lock.unlock(); }
    }

    long awaitAbove(long position, Duration timeout) throws InterruptedException {
        lock.lock();
        try {
            long nanos = timeout.toNanos();
            while (highWatermark.get() <= position && nanos > 0) {
                nanos = advanced.awaitNanos(nanos);
            }
            return highWatermark.get();
        } finally { lock.unlock(); }
    }
}

```

15.9 Observability facade

```

public interface StoreTelemetry {
    AppendObservation startAppend(AppendRequest request);
    void queueDepth(int depth);
    void storageDiagnostics(StorageDiagnostics diagnostics);
    void subscriptionLag(String consumer, long lag);
}

public interface AppendObservation extends AutoCloseable {
    void queued(Duration duration);
    void conditionChecked(String type, boolean success);
    void committed(AppendResult result, Duration walSync);
    void failed(Throwable failure);
}

```

15.10 DiagnosticService

```

public interface DiagnosticService {
    Optional<AppendDiagnosticView> appendById(String requestId);
    Optional<AppendDiagnosticView> appendByIdempotencyHash(String hash);
    Optional<ConflictDiagnosticView> conflict(String conflictId);
    EventInspection inspect(String namespace, EventLocator locator, Principal principal);
}

```

```

DiagnosticsSnapshot snapshot(DiagnosticsOptions options);
}

```

16. Testing e quality gates

16.1 Piramide di test

Livello	Obiettivo
Unit	codec, key ordering, validators, condition engine, digest canonica
Property-based	monotonicità posizioni, revisioni contigue, idempotenza, equivalenza replay
Storage integration	WriteBatch cross-CF, snapshot reads, recovery WAL, backup/restore
Concurrency	append concorrenti, DCB conflicts, overload, cancellation
Protocol	gRPC/HTTP mapping, deadlines, errors, auth
Fault injection	kill -9, disco pieno, I/O error simulato, crash tra fasi
Compatibility	apertura database creati da versioni precedenti e protocol compatibility
Performance	JMH core, db_bench/Rocks tuning, end-to-end load tests

16.2 Proprietà invarianti

- Le posizioni committate sono uniche e crescenti.
- Le revisioni di un subject sono contigue.
- Un AppendResult restituito come durable è presente dopo restart.
- Un batch è totalmente presente o totalmente assente.
- Un replay con stessa idempotency key non aggiunge eventi.
- Un replay con payload differente è rifiutato.
- Una DCB non committa se almeno una revisione non coincide.
- Gli indici ricostruiti producono gli stessi risultati delle query originali.
- Una subscription da posizione N non omette eventi che soddisfano il filtro dopo N.

16.3 Crash matrix

Punto di crash	Risultato atteso
Prima della write	Nessun evento e nessun idempotency record
Durante WAL append	Recovery RocksDB: batch interamente presente o assente
Dopo write prima della response	Retry idempotente restituisce risultato originario
Dopo commit prima del signal subscription	Timeout/poll recupera gli eventi dal log

Punto di crash	Risultato atteso
Durante backup	Backup incompleto non marcato valido
Durante rebuild index	Store originale resta utilizzabile; nuovo indice non viene attivato

16.4 Quality gates release

- Zero failure nei test invarianti e fault injection obbligatori.
- Backup e restore verificati automaticamente su dataset significativo.
- Compatibility test con almeno la release precedente.
- Benchmark pubblicato con hardware, configurazione e p50/p95/p99/p99.9.
- No unbounded queue/buffer rilevati da review.
- No payload o segreti nei log standard.
- SBOM, dependency scan e container scan completati.

17. Prestazioni e benchmark

17.1 Metriche di benchmark

- Append throughput per eventi/s e richieste/s.
- Latenza p50, p95, p99, p99.9 separando queue wait e WAL sync.
- Read throughput globale e per subject.
- Subscription catch-up throughput e live latency.
- CPU, heap allocation, native memory, block cache hit, write amplification, compaction stalls.
- Recovery time per dimensione store e backup/restore throughput.

17.2 Profili

Profilo	Descrizione
Small events	1 evento/request, payload 256 B, durable
Batch	32 eventi/request, payload 1 KiB
DCB	3, 10 e 50 consistency key/request
Hot subject	alta contesa sulla stessa revision
Mixed	70% append, 20% read, 10% subscription
Catch-up	projection da zero mentre append live continua
Compaction pressure	dataset oltre cache e write sustained

17.3 Tuning pragmatico

La configurazione RocksDB non deve essere hard-coded come “ottimale”. Il prodotto fornisce profili iniziali e metriche. Le leve principali sono block cache, write buffer, background jobs, compression, compaction e direct I/O. Ogni modifica va validata sui workload del prodotto, con particolare attenzione alla tail latency durante compaction.

18. Packaging e deployment

18.1 Artefatti

- JAR eseguibile Spring Boot.
- Container OCI con RocksDB native library inclusa.
- Java client library e BOM.
- Protobuf schema pubblicato e compatibility checked.
- CLI admin opzionale che usa le API remote.
- Helm chart single-replica con PersistentVolume e PodDisruptionBudget coerente.

18.2 Container

```
FROM eclipse-temurin:25-jre
RUN useradd --system --uid 10001 kern
COPY kern-event-store-server.jar /app/server.jar
USER 10001
VOLUME ["/var/lib/kern-event-store"]
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=50", "-jar", "/app/server.jar"]
```

Il limite heap deve lasciare memoria a RocksDB native cache, direct buffer e page cache. La configurazione di esempio è indicativa e va calcolata dal memory budget totale.

18.3 Kubernetes

- Replicas=1 nella v1; `ReadWriteOnce` PersistentVolume.
- Strategy Recreate oppure StatefulSet con prevenzione di doppio mount/split brain.
- Pod termination grace period sufficiente al drain e flush.
- Anti-affinity e backup esterno; nessuna promessa di zero downtime.
- Liveness non deve causare restart durante backup lungo; readiness separata.
- Il server deve rilevare e impedire apertura concorrente della stessa data directory.

19. Roadmap evolutiva

Fase	Contenuto	Prerequisito
0 - Prototype	Core, RocksDB layout, single append, expected revision, basic read, JMH	Semantica API approvata
1 - Production single-node	DCB key-based, durable subscription, backup/restore, auth, monitoring, fault tests	Prototype invariants stabili
1.1 - Operability	Event inspector UI/CLI, diagnostics bundle, index verify/rebuild, SLO dashboards	Production telemetry
1.2 - Persistent subscriptions	Server-side ack, competing consumers, retry policies, parking queue	Subscription semantics consolidate
1.3 - Storage evolution	Online index migration, tiered backup, optional payload compression/encryption	Format migration tooling
2 - Replication	Leader/follower, replicated commit log, follower reads, manual failover	Single-node recovery impeccabile
2.1 - Consensus	Leader election, fencing, quorum	Replication protocol validated

Fase	Contenuto	Prerequisito
	durability, automatic failover	
3 - Horizontal scale	Partitioning, per-partition position, rebalancing, routing client	Consensus e partition model
Future - Advanced DCB	Query/predicate boundaries, consistency query planner	Distributed semantics formalized
Future - Analytics	Export, CDC connectors, tiered event storage	Stable event log API

19.1 Regola evolutiva

Non introdurre clustering prima che durability, recovery, backup, compatibility e diagnosi siano affidabili nel single-node. Distribuire un motore con semantica incerta moltiplica i failure mode senza risolverli.

20. Rischi, limiti e criteri di accettazione

20.1 Rischi principali

Rischio	Mitigazione
Corruzione o incompatibilità formato	Versioning, checksum, verify, backup restore test, migration tool.
Tail latency da compaction	Metriche RocksDB, profili, benchmark sustained, alert su pending compaction/stall.
Single writer bottleneck	Group commit, batching, misurazione; sharding soltanto in roadmap.
Memoria nativa non governata	Budget esplicito heap/native/cache, metrics e startup validation.
Consumer lenti	Buffer limitati, timeout, backpressure, lag reporting.
Cardinalità osservabilità	Tag metrici controllati; dettagli in diagnostic store e trace.
Uso scorretto DCB	API tipizzata, explain conflict, documentazione key design, testkit.
False aspettative HA	Documentazione single-node, readiness corretta, backup e recovery runbook.

20.2 Criteri di accettazione v1

- Append tradizionale e DCB implementati con test di concorrenza e crash.
- ACK durable sopravvive a kill -9 e restart nei test supportati.
- Idempotenza impedisce duplicati dopo response loss.
- Read e observe ricostruiscono l'intera storia in ordine.

- Backup ripristinabile e verificato automaticamente.
- Full verify rileva indici alterati e rebuild li ricostruisce.
- Health, metriche, trace e structured logs disponibili senza payload sensibili.
- Conflict diagnostics identificano key attesa/attuale e latest position.
- Il server applica backpressure senza crescita memoria non limitata.
- Il core funziona in test senza Spring/Armeria.
- Performance report e runbook operativi inclusi nella release.

Appendice A - Errori Java

```
public sealed class EventStoreException extends RuntimeException
    permits InvalidAppendException,
             SubjectRevisionConflict,
             ConsistencyConflict,
             IdempotencyConflict,
             StoreOverloadedException,
             StorageUnavailableException,
             DataLossException {
    private final String diagnosticId;
}
```

Appendice B - Naming e convenzioni

Concetto	Convenzione
Namespace	lowercase slug, max 128 byte
Subject	URI-like o ` <code><type>:<id></code> `, max 512 byte
Event type	nome stabile versionato semanticamente, es. <code>`EnrollmentConfirmed.v1`</code>
Consistency key	chiave business esplicita, non hash casuale; namespace applicato dal server
Idempotency key	stabile per intenzione di append, max 256 byte
Tag	solo lookup esatto; evitare alta cardinalità inutile

Appendice C - Esempio end-to-end

```
// 1. Observe revisions
Revisions revisions = client.getRevisions(new RevisionQuery(
    "education",
    Set.of("enrollment:E1"),
    Set.of(new ConsistencyKey("student:S1"),
        new ConsistencyKey("course:C1"),
        new ConsistencyKey("enrollment:S1:C1"))));

// 2. Build conditional append
AppendRequest request = new AppendRequest(
    "education",
    "command:register-enrollment:E1",
    List.of(event),
    new ExpectedConsistency(revisions.consistencyKeys()),
    Set.copyOf(revisions.consistencyKeys().keySet()),
    Durability.DURABLE);

// 3. Append or receive explainable conflict
AppendResult result = client.append(request).toCompletableFuture().join();

// 4. Projection resumes from its own checkpoint
client.observe(new ObserveRequest("education", checkpoint, query), projection::apply);
```

Appendice D - Fonti tecniche di riferimento

Le decisioni tecnologiche sono coerenti con le documentazioni ufficiali: RocksDB è un key-value storage engine embedded con RocksJava, column family, viste coerenti e operazioni atomiche cross-column-family mediante WriteBatch; Armeria supporta HTTP/2, gRPC e integrazione con Spring Boot; Spring Boot Actuator e Micrometer forniscono metriche e osservabilità con integrazione OpenTelemetry. Le versioni concrete delle dipendenze devono essere fissate dal BOM del progetto e aggiornate tramite compatibility test.

- RocksDB Wiki e repository ufficiale: github.com/facebook/rocksdb
- Armeria documentation: armeria.dev/docs
- Spring Boot Actuator observability/metrics: docs.spring.io/spring-boot/reference/actuator
- Micrometer documentation: micrometer.io/docs

Appendice E - Definition of Done per capability

Capability	Done quando
Append	API, validation, atomic storage, metrics, trace, error mapping e crash test completati
DCB	multi-key expected/touched, conflict explain, concurrency property tests
Read	snapshot, cursor, index planner, limits, auth e benchmark
Observe	reconnect, backpressure, slow consumer test, lag telemetry
Backup	manifest, checksum, restore su nuova directory, scheduled verification
Diagnostics	append/conflict lookup, event inspector, redaction e retention
Release	SBOM, image, migration notes, compatibility test, runbook e benchmark report

FINE DELLA SPECIFICA v0.1